# Tu primera ventana y el bucle de juego

**Objetivo:** Al terminar puedes crear un entorno virtual, instalar pygame-ce y abrir una ventana de juego con un bucle que se puede cerrar.
**Antes de empezar:**

- Lo básico de Python - variables, if, for y funciones (lección 0 del curso).
- Linux o macOS con Python 3.10 o más nuevo. En Windows, mira la nota del resumen.
- La carpeta del curso descargada y descomprimida (trae la carpeta assets).

**Para copiar el código:** al copiar desde un PDF, muchos visores pierden los espacios del principio de cada línea, y en Python esos espacios importan. En la misma carpeta de esta guía está cada paso como archivo `.py`: ábrelo y copia de ahí.

## Paso a paso

### Tu primera ventana y el bucle de juego

Bienvenido a Python para videojuegos, donde vas a construir Most Wanted UFO Mini, un juego de naves, paso a paso. En esta lección preparas tu proyecto e instalas pygame. Después aprendes la idea más importante de cualquier juego: el bucle de juego. Y al final abres tu primera ventana, con el fondo del juego.

### Los archivos del juego están en la carpeta assets

Abre una terminal en la carpeta del curso. Ahí ya tienes una carpeta llamada assets, con los dibujos y los sonidos del juego. Escribe ls assets, y pulsa Enter, para ver qué hay dentro. Los archivos que terminan en .png son imágenes, y los que terminan en .wav son sonidos. También hay una licencia: estos recursos son de Most Wanted UFO Classic, y puedes usarlos dando crédito.

Escribe en la terminal:

```
ls assets
```

### Un entorno virtual guarda las librerías de un proyecto

Antes de instalar nada, crea un entorno virtual: una carpeta aparte donde viven las librerías de este proyecto. Escribe python3 -m venv .venv, y pulsa Enter. Ahora actívalo: escribe source .venv/bin/activate, y pulsa Enter. Fíjate que al principio de la línea aparece (.venv), entre paréntesis: el entorno está activo.

Escribe en la terminal:

```
python3 -m venv .venv
source .venv/bin/activate
```

### Pip instala una librería dentro del entorno

Con el entorno activo, instala pygame-ce: es la versión de la comunidad de pygame, y se usa igual. Escribe pip install pygame-ce, y pulsa Enter. pip descarga la librería, y la instala dentro de tu entorno. Cuando veas Successfully installed, ya está lista.

Escribe en la terminal:

```
pip install pygame-ce
```

### Crear la ventana y cargar una imagen

Ahora crea un archivo llamado game.py, en la carpeta del curso. La primera línea importa pygame: así tu programa puede usar todo lo que trae la librería, como ventanas, imágenes y sonidos. Después lo inicias con pygame.init(), que prepara la pantalla, el teclado y el sonido. Ya con pygame listo, creas la ventana, de ochocientos por seiscientos píxeles, con set_mode, y le pones un título, que se ve arriba, en la barra de la ventana. El reloj, clock, lo usarás en el bucle, para que el juego vaya a la misma velocidad en cualquier computadora. Y por último cargas la imagen del fondo, desde la carpeta assets. La ruta empieza en la carpeta donde ejecutas el juego, por eso la terminal tiene que estar en la carpeta del curso. Y convert deja la imagen lista para dibujarla rápido.

Tu `game.py` queda así (archivo `paso-1.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
```

### El bucle de juego - eventos, dibujar, repetir

Ahora viene el corazón de todo juego: el bucle. Usas una variable, running, que vale True, y un while que se repite mientras siga así. En cada vuelta, primero revisas los eventos: teclas, clics, o que alguien cierre la ventana. Si llega el evento QUIT, que significa cerrar la ventana, running pasa a False, y el bucle termina. Después dibujas: pegas el fondo con blit, en la esquina de arriba a la izquierda, y flip muestra el cuadro, es decir, la imagen nueva. Y clock.tick(60) hace que todo esto pase sesenta veces por segundo, ni más rápido ni más lento. Fíjate que la última línea ya no tiene sangría: está fuera del bucle. Al salir del bucle, pygame.quit() cierra la ventana, y libera todo lo que pygame estaba usando.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-2.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
```

```
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.blit(background, (0, 0))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

### Ejecutar el juego y cerrarlo con el evento QUIT

Guarda el archivo, y en la terminal escribe python game.py. Se abre la ventana, con el fondo de estrellas de Most Wanted UFO Mini. Todavía no se mueve nada, pero el bucle ya está corriendo sesenta veces por segundo. Para cerrarla, haz clic en la equis de la ventana: eso envía el evento QUIT, y el bucle termina.

Pruébalo:

```
python game.py
```

### Lo que aprendiste

Repasemos. Cada proyecto tiene su propio entorno virtual. Instalaste pygame-ce con pip. El bucle de juego revisa los eventos, dibuja, y se repite. Y el evento QUIT te deja cerrar la ventana. Tu ejercicio: cambia el título de la ventana, y dibuja el ovni encima del fondo. En la próxima lección, el ovni aparece, y se mueve con el teclado.

- Un entorno virtual por proyecto
- pip install pygame-ce
- El bucle: eventos, dibujar, repetir
- QUIT cierra la ventana

## Resumen

- `python3 -m venv .venv` crea un entorno virtual; `source .venv/bin/activate` lo activa. **En Windows:** `.venv\Scripts\activate`.
- Si `venv` falla en Ubuntu o Debian: `sudo apt install python3-venv`. En Windows usa `py -m venv .venv`.
- `pip install pygame-ce` instala pygame dentro del entorno activo; en el código se importa como `import pygame`.
- `pygame.display.set_mode((800, 600))` crea la ventana y `pygame.image.load(...)` carga una imagen.
- El bucle de juego se repite mientras `running` sea `True`: revisa los eventos, dibuja con `blit`, muestra el cuadro con `flip` y espera con `clock.tick(60)`.
- El evento `pygame.QUIT` llega cuando se cierra la ventana: ahí se pone `running = False`.

## Ejercicio

Parte del `game.py` de esta lección.

- Cambia el título de la ventana por uno tuyo.
- Carga `assets/player_1.png` como cargaste el fondo y dibújalo después del fondo en `(368, 480)`. ¿Qué pasa si lo dibujas antes del fondo?
- Bonus: haz que la ventana mida 1024 por 768. ¿Qué pasa con el fondo, que mide 800 por 600?

## Solución

```
pygame.display.set_caption("Mi primer juego")
ufo = pygame.image.load("assets/player_1.png").convert_alpha()
# dentro del bucle, después del fondo:
screen.blit(background, (0, 0))
screen.blit(ufo, (368, 480))
```

Si dibujas el ovni antes del fondo, el fondo lo tapa: cada `blit` pinta encima de lo anterior. Con una ventana de 1024 por 768, el fondo ocupa solo la esquina de arriba a la izquierda: el resto queda negro.

---

Gráficos y sonidos: Most Wanted UFO Classic, de Jose Daniel Soto, licencia CC BY 4.0.